

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 12 · LECTURE NOTES

Modelling

The part that decides whether any of this helps.

MODULE OUTCOMES

- Choose decision variables deliberately.
- Translate messy objectives into expressible ones.
- Handle multiple objectives honestly.
- Validate a model against the problem it represents.
- State where a model stops being valid.

1 Choosing decision variables

THE VARIABLE CHOICE DECIDES THE STRUCTURE

The same real situation can be modelled with entirely different decision variables, and the resulting formulations differ enormously in tractability (Module 11 §4) — by orders of magnitude, and sometimes by the difference between polynomial and NP-hard.

Assignment variables $x[i][j]$ versus flow variables on arcs: one formulation may have a totally unimodular constraint matrix (Module 03 §3) and solve as a linear program, while the other requires branch and bound.

Sequencing problems are the clearest case. 'The position of item i in the sequence', 'item i immediately precedes item j ', and 'item i starts at time t ' give three completely different formulations of the same problem, with different sizes, different relaxation strengths, and different symmetry.

So enumerate several variable choices before committing to one. Write each down, sketch the constraints, and consider the relaxation. **This is the highest-leverage hour in an optimisation project, and it is routinely skipped** in favour of implementing the first formulation that comes to mind.

Pattern	Meaning	What to watch for
Binary assignment	$x[i][j] = 1$ if item i is assigned to resource j .	Symmetry when the resources are identical (Module 09 §4) — $n!$ equivalent solutions unless broken.
Flow on arcs	How much of something moves along each arc.	Frequently totally unimodular, so prefer it where the problem permits — the integrality comes free (Module 11 §1).

Pattern	Meaning	What to watch for
Time-indexed	$x[i][t] = 1$ if activity i starts at time t .	Very large — one variable per activity per period — and the relaxation is strong. Frequently the better trade , because a large model with a tight bound beats a compact one with a weak bound (Module 09 §1).
Sequencing / precedence	$y[i][j] = 1$ if i precedes j .	Requires transitivity constraints, which are numerous, and the relaxation is typically weak.
Set selection	One variable per feasible set, pattern, or route.	Exponentially many variables — generate them on demand by column generation, which is Module 04's duality used as an algorithm.

2 Objectives

THE STATED OBJECTIVE IS RARELY THE REAL ONE

'Minimise cost' almost always means 'minimise cost without anything unacceptable happening' — and the set of unacceptable things is unstated, because it is obvious to the person who owns the problem and invisible to you.

So the first model produces a solution nobody will accept. That is informative rather than a failure: every objection raised is a constraint that was missing, and it is far easier to elicit an objection to a concrete solution than a specification in the abstract.

So iterate deliberately. Show the solution, collect the objections, encode them as constraints, re-solve. **The model converges on the real problem through rejection**, and three or four rounds is typical.

And a solution that is optimal and unacceptable is more useful than one that is feasible and mediocre, because the first elicits the missing constraint and the second does not. **Deliberately producing an extreme solution early is a legitimate and underused technique.**

Approach	How it works	Is it honest?
Weighted sum	Minimise $w_1 f_1 + w_2 f_2$.	It requires an exchange rate between the objectives — how many pounds is an hour of delay worth? State the rate explicitly , because the weights encode it whether or not anyone examined them.
Constrain all but one	Minimise f_1 subject to $f_2 \leq c$.	Usually the clearest formulation , because c is a quantity the problem owner can reason about directly, and the dual variable on that constraint prices the trade (Module 04 §2).
Pareto frontier	Sweep the trade-off and plot the achievable combinations.	The most honest — it shows the trade rather than hiding it in a weight , and lets the owner choose a point. Costs one solve per point.

Approach	How it works	Is it honest?
Lexicographic	Optimise the first objective, fix it, then optimise the second.	Appropriate when the priorities are genuinely strict rather than merely ordered.
Goal programming	Penalise deviation from stated targets in each dimension.	Useful when the owner has targets rather than preferences, which is common in practice.

3 Validating a model

- **Solve a tiny instance by hand and compare.** Three items and two machines, worked on paper. **The most effective check available and the least often performed**, because it feels beneath the problem.
- **Check the extremes:** zero demand, infinite capacity, a single item, identical costs. **The answers should be obvious in advance, and they should be right** — a model that gets an obvious case wrong is wrong everywhere.
- **Look at the dual variables** (Module 04 §2). **If a shadow price is absurd — an extra hour of a resource worth a million, or a binding constraint priced at zero — a constraint is wrong. This is the underused check**, and it localises the error to a specific constraint rather than merely signalling that something is off.
- **Perturb an input and check the answer moves sensibly** and in the right direction. Increase a cost and the solution should use less of that thing.
- **And show the solution to the person who owns the problem. They will spot in ten seconds what you would not find in a week** — a schedule that violates an unwritten rule, a route that passes a site that closed last year, an assignment that ignores a qualification requirement.

INFEASIBLE AND UNBOUNDED ARE BOTH DIAGNOSTICS

'Infeasible' usually means **over-constrained rather than genuinely impossible**. Use an **irreducible infeasible subset** — most commercial solvers compute one on request — to find the minimal set of mutually conflicting constraints, which is almost always a small and illuminating set.

'Unbounded' almost always means **a missing constraint**, and **it is a gift**: the solver has found a way to improve the objective without limit, **which tells you precisely what you forgot to forbid**. Reading the unbounded ray identifies the omission directly.

Add elastic variables with large penalties to constraints you suspect, so the model becomes feasible and the solution reports *which* constraint it had to violate and by how much — which turns 'infeasible' into a ranked list of problems rather than a single unhelpful word.

And a suspiciously good objective value is the same signal as a suspiciously good test score (CSCE 633 Module 12 §1). **Audit before celebrating** — the usual cause is a missing constraint that permits something the real world does not.

4 Where the model stops being the problem

STATE WHERE THE MODEL BREAKS

Every model omits things. Uncertain quantities are treated as known; human judgement is not encoded; dynamics are treated as static; rare events are excluded; and the objective is a proxy for something nobody can write down (CSCE 633 Module 01 §3 made the same point about loss functions).

And optimisation exploits every simplification aggressively, because it is explicitly searching for extremes. **A simplification that is harmless in a simulation — which samples typical behaviour — can be catastrophic in an optimum, which deliberately seeks the boundary.**

That is the specific hazard of this subject. The optimiser finds the corner of your model where it least resembles reality, and sits there, because that is where the model says the best solution is. A schedule with no slack, a portfolio concentrated in whatever the model underestimated the risk of, a route that depends on every connection being on time.

So write the assumptions down and test the solution against them. 'This model assumes demand is known exactly; under a 20% demand error the recommended solution costs 15% more than the alternative' is the deliverable — not the optimum alone. **Stochastic and robust optimisation exist to address this formally**, and even without them, a sensitivity analysis and a stated assumption list is the difference between a usable recommendation and a trap.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Williams — Model Building in Mathematical Programming	The reference for this module , and one of very few books about formulation rather than algorithms. Library copy; it is worth seeking out.
Klotz & Newman — Practical Guidelines (free)	The §3 diagnostics, including irreducible infeasible subsets and elastic formulations.
Boyd & Vandenberghe — Convex Optimization, chapters 4 and 7 (free)	Worked formulations and the multi-objective material of §2, including the Pareto frontier treatment.
Ben-Tal, El Ghaoui & Nemirovski — Robust Optimization	The formal response to §4 — optimising against the worst case within an uncertainty set rather than against a point estimate.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Formulate one problem three ways** with different decision variables, and compare size, relaxation strength, and solve time.
- 2 **Build a time-indexed and a sequencing formulation** of the same scheduling problem and compare.
- 3 **Solve a tiny instance of your project model by hand** and compare against the solver.
- 4 Check three extreme cases and verify the answers are obvious and correct.
- 5 **Inspect every dual variable** and flag any that is implausible.
- 6 **Deliberately make your model infeasible** and find the irreducible infeasible subset.
- 7 **Make it unbounded** and identify the missing constraint from the ray.
- 8 Add elastic variables and report which constraint the model most wants to violate.
- 9 **Plot the Pareto frontier** for a two-objective version of your problem.
- 10 **Write the assumption list** for your model and measure the solution's sensitivity to the two you least trust.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does the variable choice decide tractability?
- 2 Give five variable patterns and what to watch for in each.
- 3 Why is the first model's unacceptable solution useful?
- 4 Give five ways to handle multiple objectives and say which is most honest.
- 5 Give five ways to validate a model, and which is underused.
- 6 What do infeasible and unbounded each usually mean?
- 7 Why does optimisation exploit simplifications more aggressively than simulation?
- 8 What is the deliverable, beyond the optimum?